

Speech Mémoire M2

Slide 1 : page de garde

Bonjour à tous. Je m'appelle Lamia BELKADI, étudiante en Master 2 MIAGE parcours Ingénierie Logicielle pour le Web. Je suis en alternance chez ISOAR, entreprise spécialisée dans l'édition de logiciels ERP pour l'industrie. Mon mémoire porte sur l'optimisation de l'efficacité énergétique et de l'empreinte carbone des Data Centers Cloud. Je vais vous présenter aujourd'hui mon activité en entreprise, puis mon travail de recherche.

Slide 2 : plan de la présentation

Ma présentation se compose de deux parties.

La première partie présente mon entreprise : ISOAR et ses clients, mon maître d'apprentissage, les missions que j'ai réalisées, et mon évolution vers la gestion de projet.

La seconde partie porte sur mon mémoire. Je commencerai par exposer la problématique et le contexte, ainsi que les solutions existantes. Je présenterai ensuite ma solution, HEAT2VALUE : une nouvelle approche fondée sur un nouvel indicateur, le HRR, et un nouveau score environnemental. J'expliquerai son fonctionnement et son implémentation en Python. Je terminerai par les résultats obtenus, les indicateurs de performance, ainsi que les limites du projet et les axes d'amélioration possibles.

Slide 3 : synthèse entreprise

Commençons par la partie entreprise

Slide 4 : L'entreprise ISOAR et ses clients

ISOAR est une entreprise fondée en 1992, basée à Rungis. Elle est spécialisée dans l'édition de logiciels ERP dédiés à l'industrie. Son produit phare est l'ERP SQUALP — un progiciel de gestion industrielle qui intègre la qualité à l'ensemble des processus métiers de ses clients.

J'ai intégré le pôle développement informatique, qui est au cœur de l'activité de l'entreprise. Ce département assure le développement, l'évolution et la maintenance des solutions logicielles, notamment SQUALP Web, les API REST, et les applications mobiles.

Les clients d'ISOAR sont des industriels issus de secteurs très variés. On retrouve Gascogne Bois, un grand groupe européen de l'industrie du bois avec environ 3000 employés. Polytechs, spécialisé dans la fabrication de polymères. Flexelec, expert en systèmes de maintien en température. AGEMA dans le secteur du bâtiment. IMC, leader du consommable médical en Algérie. Et SOBIO, qui commercialise des produits certifiés BIO.

Slide 5 : Maître apprentissage et organisation de l'équipe

Mon maître d'apprentissage est Anthony CORTEZ, développeur principal et responsable technique chez ISOAR. Il gère le développement de l'ERP SQUALP, les projets clients et l'encadrement de l'équipe.

Comme vous pouvez le voir sur l'organigramme, l'équipe est organisée autour d'Ali ATTAB en tant que directeur, avec trois analystes programmeurs dont Anthony CORTEZ, qui encadre directement deux apprentis : Bradley MUTIMA et moi-même.

Tout au long de mon alternance, il m'a confié des missions progressives et responsabilisantes, avec un suivi régulier qui m'a permis de monter en compétences rapidement.

Slide 6 : Les travaux réalisés chez isoar

Au cours de mon alternance, j'ai travaillé sur quatre projets.

SQUALP Web : j'ai adapté l'ERP vers une version web et mobile. J'ai développé des modules comme le transfert de stocks, les inventaires et les ordres de services, avec des interfaces réactives, un scanner de codes-barres et des fenêtres modales pour la gestion d'articles.

Le projet IXAO : un système de gestion des demandes. J'ai apporté des améliorations front-end et back-end, et ajouté de nouvelles fonctionnalités pour améliorer l'expérience utilisateur.

L'API REST pour Gascogne Bois : j'ai développé des API en JavaScript et PHP pour le module des commandes, en utilisant des procédures stockées SQL et en validant avec Postman.

Et ma mission actuelle : les Statistiques Ventes. Je développe un module de visualisation de données avec Chart.js — graphiques interactifs, comparaison annuelle CA/Coût/Marge, export PDF, avec des données récupérées via API REST et SQL Server.

Slide 7 : Evolution vers gestion de projet

Au fil de l'alternance, mon rôle a évolué au-delà du développement pur. J'ai progressivement pris des responsabilités en gestion de projet aux côtés de mon maître d'apprentissage : analyse des besoins clients, priorisation des tâches, suivi des projets et coordination entre les différentes phases de développement. Cette évolution m'a donné une vision plus globale du cycle de vie d'un projet informatique en entreprise.

Sur le plan technique, j'ai travaillé principalement en JavaScript, PHP et SQL. J'ai également lu et compris du code Delphi pour mieux m'intégrer dans l'existant. La base de données utilisée est Microsoft SQL Server.

Côté outils, j'ai utilisé Git pour la gestion de versions, VS Code comme éditeur, XAMPP pour l'environnement local, Postman pour tester les API, SQL Server Profiler pour analyser les requêtes, et Trello pour organiser les tâches et suivre l'avancement des projets.

Slide 8 : Présentation du mémoire

Passons maintenant à la présentation de mon mémoire.

Slide 9 : Introduction et problématique

Les Data Centers sont partout autour de nous, même si on ne les voit pas. Chaque fois qu'on envoie un email, qu'on regarde une vidéo ou qu'on utilise une application, des milliers de serveurs travaillent en permanence pour rendre ça possible.

Ces serveurs consomment des quantités énormes d'électricité. Et ce que beaucoup de gens ignorent, c'est que 95% de cette électricité ne disparaît pas — elle se transforme en chaleur. Une chaleur qui est aujourd'hui simplement rejetée dans l'atmosphère, sans être valorisée.

Avec l'essor de l'intelligence artificielle, la situation s'aggrave encore. Les GPU utilisés pour l'IA consomment jusqu'à trois fois plus qu'un serveur classique, ce qui pousse les Data Centers à leurs limites.

C'est face à ce constat que j'ai posé la problématique de mon mémoire :

Comment optimiser la performance énergétique des Data Centers Cloud afin de réduire durablement leur empreinte carbone ?

Slide 10 : état de l'art et ses limites

Pour répondre à cette problématique, j'ai d'abord analysé ce que la recherche scientifique propose. L'état de l'art identifie quatre grandes stratégies.

La première est l'**optimisation logicielle**. L'idée est simple : plutôt que d'éteindre un serveur qui ne travaille pas, on réduit intelligemment sa consommation en baissant la tension et la fréquence du processeur selon la charge. C'est ce qu'on appelle le Power Capping et le DVFS.

La deuxième est la **flexibilité géographique**. Si un Data Center en France utilise de l'électricité polluante à un moment donné, le système déplace automatiquement les calculs vers un pays où l'électricité est plus propre. C'est le principe "Follow the Sun".

La troisième est l'**indépendance énergétique**. On couple directement des éoliennes ou des panneaux solaires au Data Center, avec des batteries pour stocker l'énergie et s'affranchir du réseau électrique classique.

La quatrième est la **gestion holistique**, avec SHIELD — un framework qui optimise simultanément les trois indicateurs : l'énergie, le carbone et l'eau.

Ces solutions sont intéressantes. Mais elles partagent toutes la même limite, visible en bas du tableau : **la chaleur produite par les serveurs est toujours traitée comme un déchet à éliminer, jamais comme une ressource à valoriser.**

C'est cette limite qui m'a conduite à proposer une approche différente.

Slide 11 : HEAT2VALUE changement de paradigme

C'est exactement là que j'ai voulu apporter quelque chose de nouveau.

Toutes les solutions que je viens de présenter cherchent à mieux gérer la chaleur — à l'évacuer plus efficacement, à consommer moins pour la refroidir. Mais personne ne s'est posé la question suivante : et si on utilisait cette chaleur plutôt que de la jeter ?

Aujourd'hui, la logique est simple mais coûteuse : les serveurs consomment de l'électricité, produisent de la chaleur, et on paie une deuxième fois pour s'en débarrasser via des systèmes de climatisation. C'est du gaspillage à double niveau.

Avec HEAT2VALUE, cette chaleur peut être envoyée vers des bâtiments voisins pour les chauffer, stockée dans un réservoir thermique pour une utilisation ultérieure, ou rejetée en dernier recours seulement si aucune autre option n'est disponible.

Ce changement de paradigme permet de réduire simultanément la consommation de refroidissement, les émissions de carbone et la consommation d'eau. Et pour mesurer cette valorisation, j'ai introduit un nouvel indicateur que je vais vous présenter maintenant.

Slide 12 : heat2value vs solutions existantes

Ce tableau résume le positionnement de HEAT2VALUE par rapport aux solutions existantes. On voit que SHIELD, WaterWise, Carbon-Aware Scheduling et le refroidissement par liquide couvrent chacun une partie des indicateurs classiques — PUE, CUE et WUE. Mais aucune ne valorise la chaleur. HEAT2VALUE est la seule solution à cocher les quatre cases simultanément.

Slide 13 : le nouveau kpi : HRR

Pour mesurer cette valorisation, j'ai introduit un nouveau KPI que j'appelle le **Heat Reuse Ratio**, ou HRR. Sa formule est simple :

$$\text{HRR} = \text{Chaleur réutilisée} / \text{Chaleur totale produite}^{**}$$

Ce qui rend ce KPI original, c'est qu'il comble un vide que personne n'avait remarqué. Le PUE, le CUE et le WUE mesurent tous ce qu'on consomme ou ce qu'on pollue. Aucun ne mesure ce qu'on valorise. Un Data Center qui chauffe un immeuble voisin en remplaçant sa chaudière à gaz évite des émissions de CO₂ réelles — mais cet impact positif est complètement invisible dans les indicateurs existants. Le HRR le rend visible et mesurable.

En pratique : un HRR de 0 correspond à un Data Center classique qui rejette tout. Un HRR de 0.80 signifie que 80% de la chaleur est valorisée — c'est le résultat que j'obtiens en conditions hivernales. Et un HRR de 1 serait le cas idéal où toute la chaleur est réutilisée.

Slide 14 : le nouveau score environnemental globale

Pour intégrer le HRR dans une vision globale, j'ai créé un nouveau score environnemental.

Jusqu'ici, les travaux existants combinaient déjà les trois KPI classiques dans un score global : PUE, CUE et WUE. Mais ce score ne tenait pas compte de la valorisation de la chaleur. J'ai donc étendu ce score en y ajoutant le HRR :

$$\text{Score H2V} = \alpha \cdot \text{PUE} + \beta \cdot \text{CUE} + \gamma \cdot \text{WUE} - \delta \cdot \text{HRR}$$

Les coefficients α , β et γ sont les poids accordés aux trois KPI classiques. Ils sont ajustables selon le contexte : par exemple, dans une région en stress hydrique, on augmente le poids du WUE. Le coefficient δ est le poids accordé à la valorisation thermique. Et le HRR est **soustrait** intentionnellement : plus on valorise, plus le score baisse, ce qui signifie une meilleure performance environnementale globale.

C'est donc une extension naturelle de ce qui existait déjà, avec une quatrième dimension originale qui n'avait jamais été intégrée auparavant.

Slide 15 : données d'entrées

Pour prendre ses décisions, l'algorithme collecte trois types de données toutes les 15 minutes.

Les données internes du Data Center : la charge des serveurs. Plus les serveurs travaillent, plus ils consomment d'électricité, et plus ils produisent de chaleur. C'est le point de départ de toute décision.

Les données météorologiques : la température extérieure. C'est la donnée la plus importante — si la température est inférieure à 18°C, les bâtiments voisins ont besoin de chauffage, et la chaleur peut être valorisée immédiatement.

Les données énergétiques : l'intensité carbone du réseau électrique, c'est-à-dire la quantité de CO₂ émise pour produire chaque kWh. Si cette valeur dépasse 150 gCO₂/kWh, cela signifie que le réseau est polluant, et il vaut mieux stocker la chaleur plutôt que d'utiliser encore plus d'électricité pour refroidir.

Slide 16 : les options de destination de chaleur

À partir de ces trois données, l'algorithme choisit à chaque instant l'une des trois options. S'il fait froid dehors, il réutilise la chaleur pour chauffer — 80% est valorisée. Si le carbone est élevé et que le réservoir n'est pas encore plein, il stocke la chaleur — 70% est stockée. Et si aucune de ces conditions n'est remplie, il rejette la chaleur — c'est le dernier recours, et c'est exactement ce que font tous les Data Centers classiques aujourd'hui.

Slide 17 : Moteur de décision

Ce schéma résume en une image toute la logique de l'algorithme.

À chaque évaluation — toutes les 15 minutes — l'algorithme pose deux questions simples dans un ordre précis.

Première question : est-ce que la température extérieure est inférieure à 18°C ? Si oui, les bâtiments ont besoin de chauffage, et on valorise immédiatement la chaleur. C'est l'option prioritaire car c'est la plus efficace et la plus directe.

Si la réponse est non, on passe à la deuxième question : est-ce que l'intensité carbone du réseau dépasse 150 gCO₂/kWh, et le réservoir n'est pas encore plein ? Si oui, on stocke la chaleur pour l'utiliser plus tard, quand les conditions seront meilleures.

Et si les deux réponses sont non, seulement à ce moment-là, on rejette la chaleur. Le rejet n'est jamais un choix — c'est un dernier recours.

Cette logique est intentionnellement simple. Un algorithme clair est plus facile à déployer, à maintenir et à faire évoluer dans un contexte industriel réel.

Slide 18 : illustration par des scénarios concrets

Pour illustrer concrètement le comportement de l'algorithme, j'ai testé trois scénarios représentatifs.

En hiver, la température reste constamment en dessous de 18°C. L'algorithme choisit donc toujours l'option A — la réutilisation immédiate. La chaleur des serveurs chauffe directement des bâtiments voisins, en remplaçant leur chaudière à gaz. Résultat : les quatre indicateurs s'améliorent simultanément, et le HRR atteint **0.80** — 80% de la chaleur est valorisée au lieu d'être rejetée.

En été, la température dépasse 18°C toute la journée. Personne n'a besoin de chauffage, donc la réutilisation est impossible. Et en France, le carbone du réseau est très bas grâce au nucléaire, donc le stockage n'est pas non plus déclenché. L'algorithme rejette tout — le HRR tombe à **0**. C'est la limite saisonnière de HEAT2VALUE, que j'assume honnêtement.

En période de transition, comme l'automne ou le printemps, la température oscille autour de 18°C. Le matin il fait frais, l'algorithme réutilise. En journée la température monte, il bascule vers le stockage ou le rejet. Le soir il refroidit, il revient à la réutilisation. C'est le scénario le plus riche : les trois options s'activent dans la même journée, et le HRR se situe entre **37% et 78%** selon les conditions.

Slide 19 : python et bibliothèques

Pour l'implémentation, j'ai choisi Python. C'est le langage de référence de la communauté scientifique en optimisation des infrastructures numériques, et il est cohérent avec les pratiques de recherche que j'ai étudiées, comme SHIELD, Carbon-Aware ou le Deep Reinforcement Learning. Python offre aussi un écosystème riche de bibliothèques scientifiques, tout en restant accessible, lisible et facilement maintenable — un point essentiel pour un déploiement industriel.

Trois bibliothèques structurent l'algorithme. NumPy prend en charge les calculs mathématiques, comme les formules du HRR, du Score H2V et des indicateurs classiques. Pandas gère les données temporelles, avec des mesures prises toutes les 15 minutes sur 24 heures. Et Matplotlib permet de visualiser les résultats, notamment les graphiques de comparaison avant/après HEAT2VALUE.

Slide 20 : architecture modulaire

L'implémentation est organisée en cinq modules Python indépendants. Le simulateur génère les données du Data Center. Le module de décision applique l'algorithme HEAT2VALUE. Le module KPI calcule les quatre indicateurs. Le module de comparaison compare les résultats avec et sans HEAT2VALUE. Enfin, les modules de tests vérifient l'algorithme sur des données réelles et des scénarios variés.

Slide 21 : choix de simulation

J'ai choisi la simulation parce qu'un Data Center est un environnement critique : on ne peut pas tester un algorithme en conditions réelles sans risque. Pour rendre les tests plus crédibles, j'ai utilisé de vraies données d'intensité carbone, fournies par Electricity Maps, une plateforme qui publie ces données heure par heure sur toute l'année 2024. J'ai aussi créé 12 scénarios construits à la main pour couvrir des situations variées.

Slide 22 : Simulateur de data center

Le simulateur génère 96 points de données sur 24 heures, avec une mesure toutes les 15 minutes. La puissance maximale est de 1000 kW, et la chaleur produite est calculée à partir de la puissance consommée. Ici, un extrait des données produites sur le scénario du 15 janvier 2024.

Slide 23 : Le module de décision

Le module de décision évalue les 96 instants de la journée à partir de trois paramètres fixes : seuil de température, seuil de carbone, et capacité du réservoir. Ici, sur ce même scénario hiver, on voit que la réutilisation est choisie à chaque instant, avec la chaleur valorisée à droite.

Slide 24 : le module calcule kpi

Le module KPI calcule trois indicateurs classiques du secteur. Le PUE dépend de la décision prise, entre réutilisation, stockage et rejet. Le CUE correspond aux émissions totales moins celles évitées grâce à la valorisation de la chaleur. Et le WUE dépend directement de la décision. Ici, sur ce même scénario, on voit ces trois KPI calculés à chaque instant.

Slide 25 : résultats comparaison

Voici les résultats obtenus sur le scénario de référence hivernal. La comparaison est très claire : le PUE passe de 1.30 à 1.05, soit une réduction de 25%. Le CUE est réduit de 80% grâce aux émissions évitées par la valorisation. Le WUE passe de 2.0 à 0.5 L/kWh, soit 75% de réduction de la consommation d'eau. Et le HRR atteint 0.80 — 80% de la chaleur est valorisée au lieu d'être rejetée.

Slide 26 : graphique comparaison

Le graphique confirme visuellement ces résultats. On voit clairement que la ligne verte reste nettement sous la rouge pour le CUE, et que le PUE et le WUE sont constants et améliorés toute la journée.

Slide 27 : résultats electricity maps

Sur les données réelles d'Electricity Maps pour trois journées de 2024, les résultats sont cohérents. Le 15 janvier en hiver : température entre 2 et 10°C, réutilisation systématique, HRR de 80% et CUE nul. Le 15 juillet en été : température élevée, carbone français très bas à 4.75 gCO₂/kWh, rejet total — c'est la limite saisonnière confirmée. Le 30 décembre : température froide, réutilisation systématique, mêmes excellents résultats qu'en janvier.

Une observation importante : le réseau électrique français est exceptionnellement propre — entre 4 et 50 gCO₂/kWh — grâce au nucléaire. HEAT2VALUE serait encore plus pertinent dans des pays avec un mix énergétique plus carboné, comme l'Allemagne ou la Pologne.

Slide 28 : resultats 12 scénarios

Sur les 12 scénarios testés, trois résultats se dégagent. En saison froide, sept scénarios sur douze atteignent un HRR de 80 %, avec les meilleurs KPI : la chaleur est valorisée de façon systématique. Le scénario de transition automne est le plus riche : les trois options — réutilisation, stockage, rejet — sont actives dans la même journée, avec un HRR intermédiaire de 37 %. À l'inverse, les quatre scénarios estivaux montrent la limite saisonnière de l'algorithme, avec un HRR nul.

Slide 29 : synthèses des resultats

En synthèse, par rapport à la référence sans HEAT2VALUE : le PUE baisse de 19 % dès que la réutilisation est possible. Le CUE tend vers zéro en hiver, avec un réseau propre, et reste positif mais réduit en été. Le WUE diminue nettement en mode réutilisation. Et le HRR atteint 0,80 en hiver, entre 37 et 78 % en transition, et 0 en été. La performance globale est mesurée par le Score H2V — un score bas signifie une meilleure performance environnementale.

Slide 30 : Les limites

Je veux être honnête sur les limites de mon travail.

D'abord, les limites de l'algorithme. La première limite est la saison. L'algorithme valorise la chaleur quand il fait froid et quand le carbone est élevé. Mais en été, ces deux conditions n'arrivent presque jamais. Il fait trop chaud, et le réseau électrique est déjà propre. Donc l'algorithme ne trouve rien à valoriser pendant cette période.

Ensuite, il y a une limite géographique. L'algorithme est pensé pour les pays froids, où on a besoin de chaleur toute l'année. Dans un pays chaud, il serait beaucoup moins utile.

Il y a aussi une limite liée au réseau électrique. En France, l'électricité vient beaucoup du nucléaire, donc elle est déjà peu carbonée. Le critère carbone déclenche donc rarement le stockage. Dans un pays qui utilise du charbon, l'algorithme se comporterait différemment.

Enfin, les seuils de décision — la température, le carbone, la capacité du réservoir — sont fixes. Ils ne s'adaptent pas automatiquement à un site réel. Il faudrait les régler à la main selon le contexte.

Maintenant, les limites de ma validation.

J'ai simulé la charge des serveurs de façon simple, avec une progression linéaire. En réalité, la charge changerait plus souvent, avec des pics soudains. Ma simulation est donc plus régulière que la réalité.

Ensuite, les chiffres que j'ai utilisés — pour les pertes, le refroidissement, l'eau — viennent de la littérature scientifique. Ils ne viennent pas d'un vrai site mesuré. Ils donnent un bon ordre de grandeur, mais pas une précision garantie pour un site en particulier.

Enfin, je suppose que l'infrastructure pour valoriser la chaleur existe déjà — le réseau de distribution, le réservoir de stockage, les connexions. En réalité, construire cette infrastructure représenterait un investissement important.

Slide 31 : perspectives d'amélioration

Ces limites ouvrent cinq perspectives concrètes.

D'abord, intégrer des prévisions météorologiques, pour stocker la chaleur en avance plutôt que réagir après coup. Ensuite, élargir les usages en été, avec l'eau chaude sanitaire, la climatisation par absorption, ou le séchage industriel — cela réduirait la limite saisonnière. Troisième piste : rendre les seuils adaptatifs, en les ajustant selon le pays, la saison, ou le prix de l'électricité.

Je pourrais aussi tester l'algorithme dans des pays au réseau plus carboné, comme l'Allemagne ou la Pologne, pour valider son efficacité dans un contexte énergétique différent. Et enfin, déployer un pilote réel, dans un Data Center universitaire ou une PME, à petite échelle, pour affiner les paramètres en conditions réelles.

Slide 32 : conclusion

Pour conclure, HEAT2VALUE répond à la question de mon mémoire. En valorisant la chaleur des serveurs plutôt qu'en la jetant, l'algorithme réduit en même temps trois indicateurs : le PUE, le CUE et le WUE. En hiver, on obtient -19 % sur le PUE, -80 % sur le CUE, et -75 % sur le WUE.

Le HRR et le Score H2V donnent une vision unifiée et nouvelle de la performance environnementale. Ces indicateurs n'existaient pas avant dans la littérature.

Mes résultats sont cohérents sur trois types de validation : les données simulées, les données réelles, et douze scénarios variés.

Enfin, les limites identifiées — la saison et le contexte géographique — ouvrent des pistes concrètes pour la recherche et pour un vrai déploiement.

Slide 33 : Merci

Je vous remercie pour votre attention et reste disponible pour répondre à vos questions.